

Lesson 19: Inheritance - An Overview

The ability to use the object-oriented programming is an important feature of C++. [Lesson 12](#) introduced the idea of the class; if you have not read it and do not know the basic details of classes, you should read it before continuing this tutorial. This tutorial is n Inheritance is an important feature of classes; in fact, it is integral to the idea of object oriented programming. Inheritance allows you to create a hierarchy of classes, with various classes of more specific natures inheriting the general aspects of more generalized classes. In this way, it is possible to structure a program starting with abstract ideas that are then implemented by specific classes. For example, you might have a class Animal from which class dog and cat inherent the traits that are general to all animals; at the same time, each of those classes will have attributes specific to the animal dog or cat.

Inheritance offers many useful features to programmers. The ability, for example, of a variable of a more general class to function as any of the more specific classes which inherit from it, called polymorphism, is handy. For now, we will concentrate on the basic syntax of inheritance. Polymorphism will be covered in its own tutorial.

Any class can inherit from any other class, but it is not necessarily good practice to use inheritance (put it in the bank rather than go on a vacation). Inheritance should be used when you have a more general class of objects that describes a set of objects. The features of every element of that set (of every object that is also of the more general type) should be reflected in the more general class. This class is called the base class. base classes usually contain functions that all the classes inheriting from it, known as derived classes, will need. base classes should also have all the variables that every derived class would otherwise contain.

Let us look at an example of how to structure a program with several classes. Take a program used to simulate the interaction between types of organisms, trees, birds, bears, and other creatures coinhabiting a forest. There would likely be several base classes that would then have derived classes specific to individual animal types. In fact, if you know anything about biology, you might wish to structure your classes to take advantage of the biological classification from Kingdom to species, although it would probably be overly complex. Instead, you might have base classes for the animals and the plants. If you wanted to use more base classes (a class can be both a derived of one class and a base of another), you might have classes for flying animals and land animals, and perhaps trees and scrub. Then you would want classes for specific types of animals: pigeons and vultures, bears and lions, and specific types of plants: oak and pine, grass and flower. These are unlikely to live together in the same area, but the idea is essentially there: more specific classes ought to inherit from less specific classes.

Classes, of course, share data. A derived class has access to most of the functions and variables of the base class. There are, however, ways to keep derivedren from accessing some attributes of its base class. The keywords public, protected, and private are used to control access to information within a class. It is important to remember that public, protected, and private control information both for specific instances of classes and for classes as general data types. Variables and functions designated public are both inheritable by derived classes and accessible to outside functions and code when they are elements of a specific instance of a class. Protected variables are not accessible by functions and code outside the class, but derived classes inherit these functions and variables as part of their own class. Private variables are neither accessible outside the class when it is a specific class nor are available to derived classes. Private variables are useful when you have variables that make sense in the context of large idea.